

PADRÕES DE PROJETO (DESIGN PATTERNS)

Enquanto paradigmas agem majoritariamente na etapa de **implementação**, padrões de design atuam principalmente na etapa de **projeto**.

Por que Padrões?

- Desenvolvedores acumulam soluções para os problemas que resolvem com **frequência**;
- Essas soluções são difíceis de serem elaboradas e podem aumentar a **produtividade, qualidade e uniformidade do software**.

ELEMENTOS ESSENCIAIS DE UM PADRÃO (4 elementos)

- **Nome**
Para **referenciar** o padrão (em uma ou duas palavras).
- **Problema**
Explica o **problema** e o **contexto** para que possamos identificar **quando aplicá-lo**.
- **Solução (SRRC)**
Descreve os elementos da **solução, relacionamentos, responsabilidades e colaborações**.
- **Consequências**
Resultados e trade-offs aplicados ao padrão.

PRINCIPAIS PADRÕES DE PROJETO

- **Padrões de Criação**: trata da **criação** de objetos (ex: Singleton, Abstract Factory);
- **Padrões Estruturais**: cuida da **composição** de **classes** e **objetos** (ex: Facade, Adapter, Decorator);

- **Padrões Comportamentais:** caracteriza o **modo como** as classes e objetos interagem e distribuem responsabilidades (ex: Observer, Strategy).

PRÓS E CONTRAS

- **Prós**

- **Facilitam** o re-uso de **técnicas** e **arquiteturas** de software já testadas com sucesso;
- Define um **padrão de comunicação** compartilhado pela equipe de desenvolvimento;
- Aumento de produtividade;
- Uniformidade na estrutura do software;
- Aplicação imediata por outros desenvolvedores;
- Redução da complexidade: **blocos construtivos**.

- **Contras**

- Precisa ser reprogramado em cada aplicação;
- Uso indiscriminado pode aumentar muito a complexidade.

SINGLETON

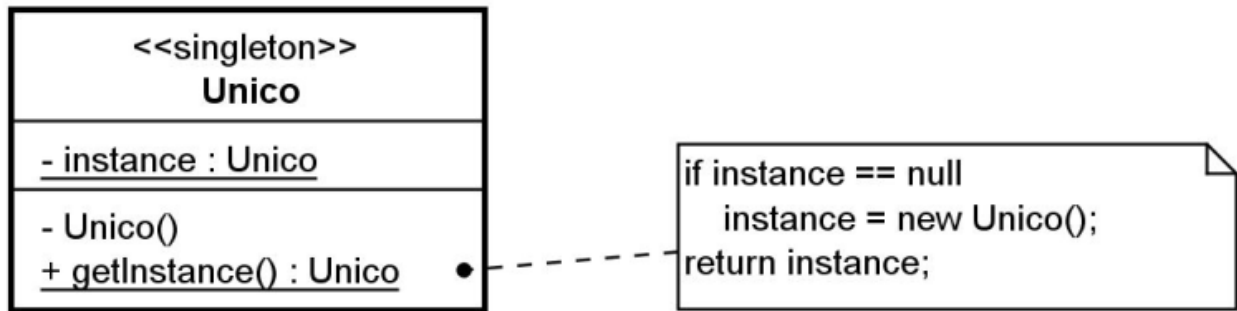
Única instância com acesso global.

Intenção: Garantir que uma classe tenha somente uma instância e fornecer um **ponto de acesso global** de acesso para a mesma.

Aplicabilidade: Quando for preciso haver apenas uma instância de uma classe, e essa instância tiver que dar acesso aos clientes através de um ponto bem conhecido. Quando a única instância tiver de ser extensível através de subclasses, possibilitando aos clientes usar uma instância estendida sem alterar seu código.

A **própria classe** é responsável pela **criação de uma única instância** e **por fornecer um meio para acessar** essa instância. A classe Singleton também define um método ***getInstance()*** para acesso à instância única, verificando se a mesma já existe, criando-a se for necessário.

Características: Classe ***static***, única forma de acesso é por meio do ***getInstance()***, **construtor privado** e propriedade ***instance*** guarda referência à instância.



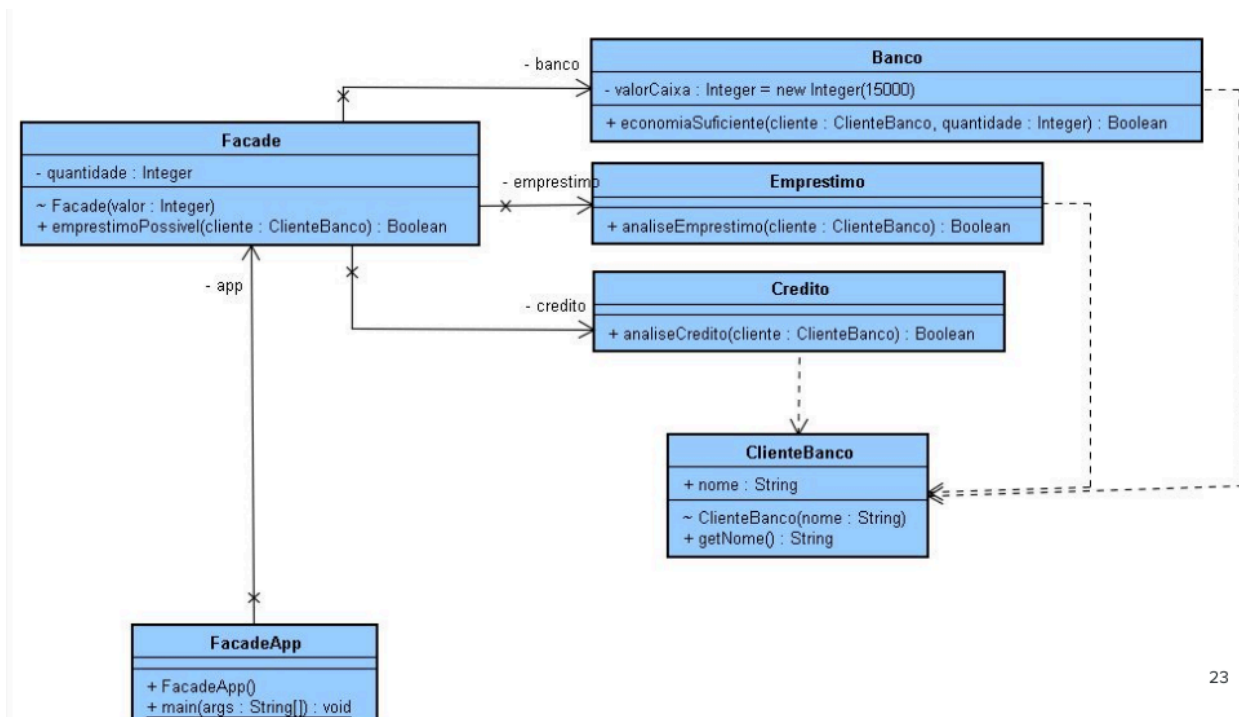
FACADE

Interface simplificada para subsistema complexo.

Intenção: Fornecer uma interface unificada para um conjunto de interfaces em um subsistema.

Aplicabilidade: Quando deseja fornecer uma interface simples para um subsistema complexo. Quando existem muitas dependências entre clientes e classes de implementação de uma abstração (promove a independência e portabilidade dos subsistemas).

Implementar um Facade demanda **definir um conjunto de operações reduzidas** que permita **ocultar a complexidade** inerente à utilização de várias classes de um subsistema.



```

public class Facade {
    private Banco banco;
    private Emprestimo emprestimo;
    private Credito credito;

    public Facade() {
        this.banco = new Banco();
        this.emprestimo = new Emprestimo();
        this.credito = new Credito();
    }

    public Boolean emprestimoPossivel(ClienteBanco cliente, Integer quantidade) {
        boolean caixaDisponivel = banco.economiaSuficiente(cliente, quantidade);
        boolean emprestimoAprovado = emprestimo.analiseEmprestimo(cliente);
        boolean creditoAprovado = credito.analiseCredito(cliente);

        return caixaDisponivel && emprestimoAprovado && creditoAprovado;
    }
}

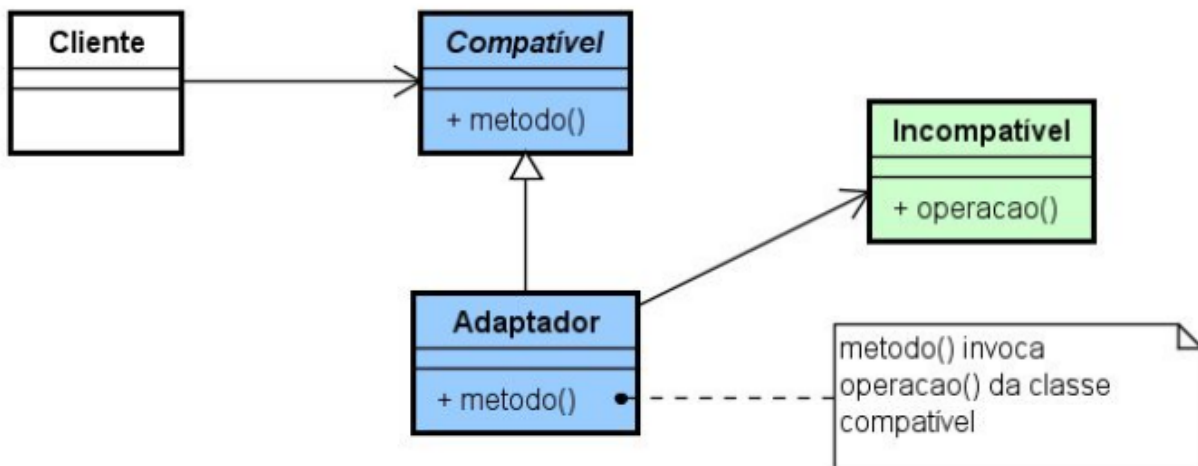
```

ADAPTER

Converte uma interface incompatível em uma interface esperada.

Intenção: Permitir que classes com interfaces incompatíveis possam trabalhar em conjunto.

Aplicabilidade: Quando quiser usar uma classe existente, mas sua interface não corresponder à interface de que necessita. Quando quiser criar uma classe reutilizável que coopere com classes não-relacionadas ou não-previstas, ou seja, classes que não necessariamente tenham interfaces compatíveis.

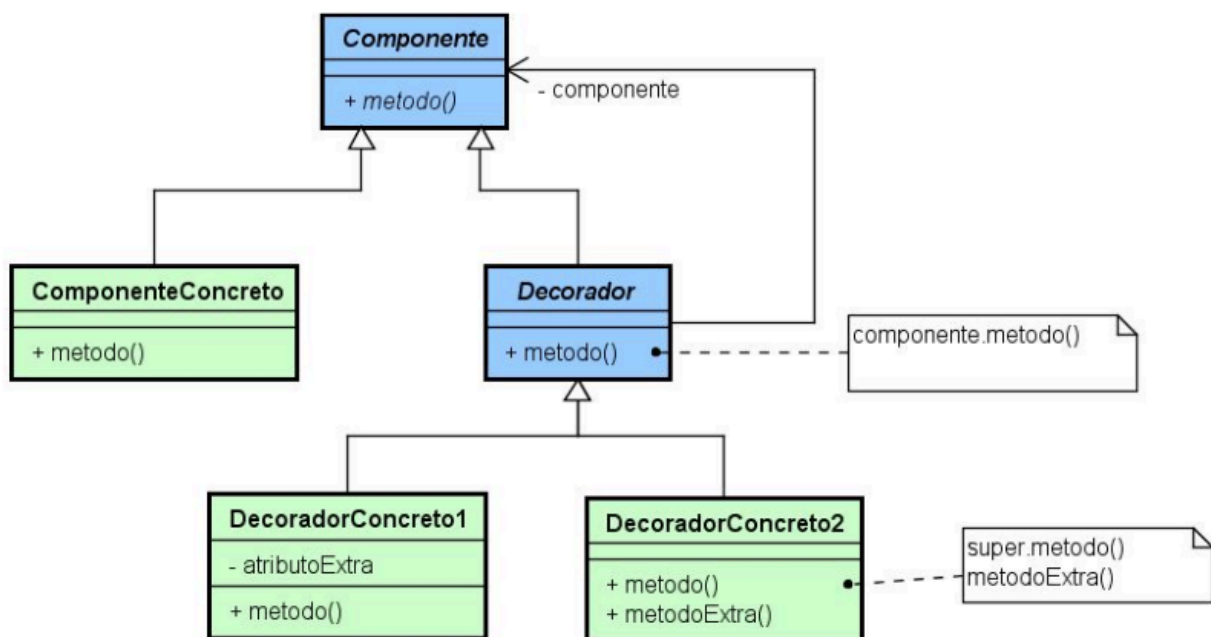


DECORATOR

Adicionando responsabilidades dinamicamente a objetos.

Intenção: Dinamicamente, **agregar responsabilidades** adicionais a um **objeto**. Os decoradores fornecem uma alternativa flexível ao uso de subclasses para extensão de funcionalidades.

Aplicabilidade: Para acrescentar responsabilidades a objetos individuais de forma dinâmica e transparente, ou seja, **sem afetar outros objetos**. Para responsabilidades que podem ser **removidas**. Quando a extensão através do uso de subclasses não é prática (grande número de subclasses).



OBSERVER

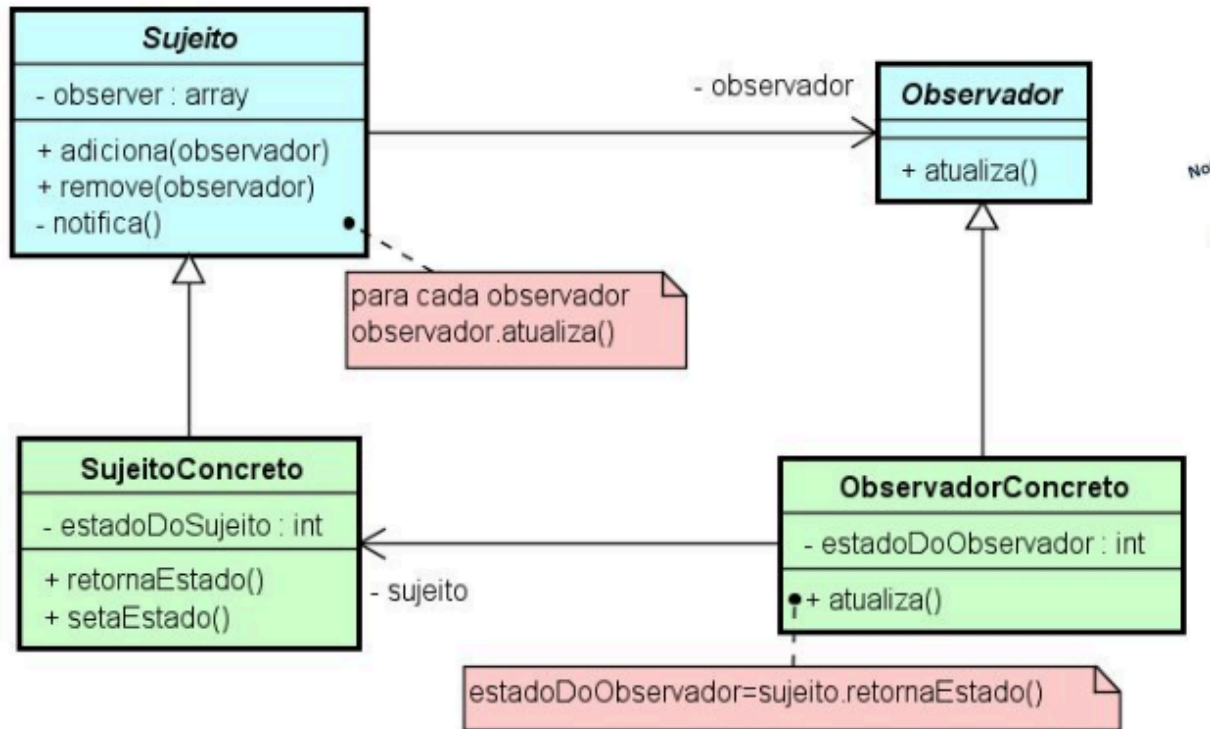
Notifica dependentes sobre mudanças de estado.

Intenção: Definir uma dependência um-para-muitos entre objetos, de maneira que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.

Aplicabilidade: Quando uma abstração tem dois aspectos, um depende do outro, encapsulando esses aspectos em objetos separados, permite-se variá-los e reutilizá-los independentemente. Uma mudança em um objeto exige mudanças em outros objetos, e não se sabe quantos objetos necessitam ser mudados. Um objeto

deveria ser capaz de notificar outros objetos sem fazer hipóteses, ou usar informações sobre quem são esses objetos.

Objeto observável alerta com uma chamada **notifica()**, que executa **atualiza()** em cada observador.



COMMAND

Encapsula requisição como objeto.